

REGISTRATION WEB APPLICATION

2-Tier Architecture Using AWS EC2 (Frontend + Backend)

AWS Batch : July

Batch Timing : 11:30 AM to 1:00 PM

Frontend : NGINX (HTML, CSS)

Backend : Flask API (python)

INTRODUCTION:

The Registration Web Application is a web-based application developed to collect and process user information such as name and email address. The application provides a user-friendly interface where users enter their details and submit the registration form. The frontend of the application is developed using HTML and CSS, while the backend is developed using python Flask.

OBJECTIVES:

- To develop a simple and user-friendly registration web application.
- To collect user information such as name and email address.
- To connect the frontend with a Python Flask backend.
- To process and display the submitted user information.
- To demonstrate API integration using JavaScript.

AWS Infrastructure Setup

Step 1: Create VPC

- Create a VPC with CIDR block, e.g., 10.0.0.0/20.

Step 2: Create Subnets

- Public Subnet: For frontend EC2 (Nginx), e.g., 10.0.16.0/24.
- Private Subnet: For backend EC2 (Flask), e.g., 10.0.126.0/26.

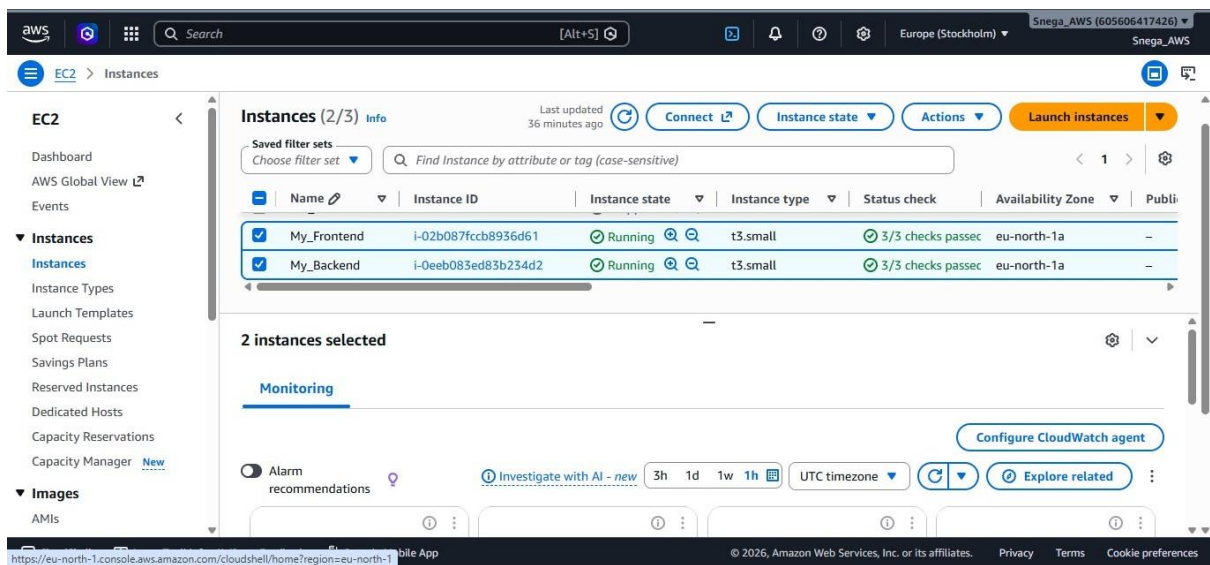
Step 3: Internet Gateway & Route Tables

- Attach IGW to the VPC.
- Public subnet route table:
 - Destination: 0.0.0.0/0 → Target: IGW
- Private subnet route table:
 - Destination: 0.0.0.0/8 → Target: NAT Gateway
 - Destination: 0.0.0.0/0 → Target: IGW

Step 4: Launch EC2 Instances

- WEBFRONT(NGINX)
- Public subnet
- Ubuntu AMI
- User Data (NGINX INSTALLED AND STARTED, Automatic Run)
- APPSERVER
- Private subnet , Auto assign IP(enable)
- Ubuntu AMI

EC2 instances for Frontend and Backend:

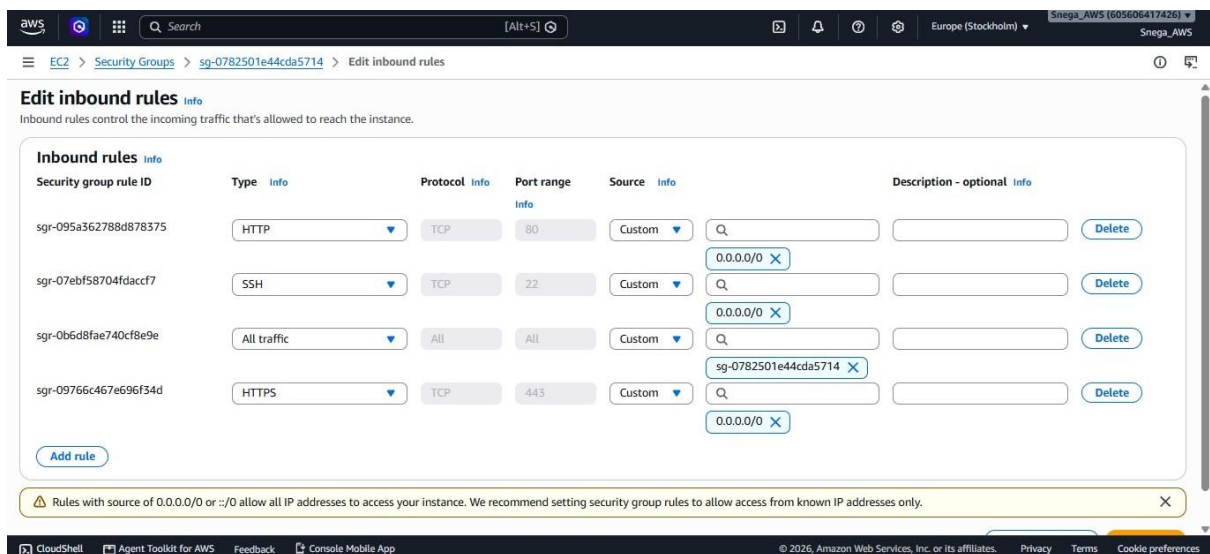


The screenshot shows the AWS Management Console for the 'Sneha_AWS' account in the 'Europe (Stockholm)' region. The 'Instances' page displays two EC2 instances:

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
My_Frontend	i-02b087fccb8936d61	Running	t3.small	3/3 checks passed	eu-north-1a	-
My_Backend	i-0eeb083ed83b234d2	Running	t3.small	3/3 checks passed	eu-north-1a	3.120.10.10

Below the table, the 'Monitoring' section shows '2 instances selected'. There are buttons for 'Configure CloudWatch agent', 'Alarm recommendations', 'Investigate with AI - new', and 'Explore related'.

Frontend Security Group:

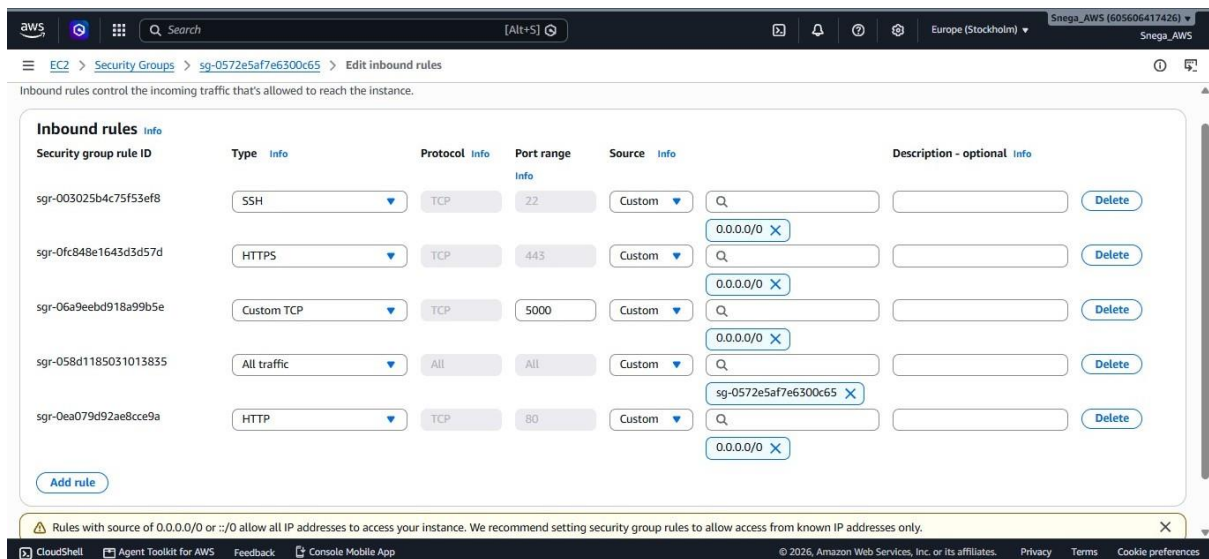


The screenshot shows the 'Edit inbound rules' page for the security group 'sg-0782501e44cda5714'. The page lists four inbound rules:

Security group rule ID	Type	Protocol	Port range	Source	Description - optional
sgr-095a362788d878375	HTTP	TCP	80	0.0.0.0/0	
sgr-07ebf58704fdacdf7	SSH	TCP	22	0.0.0.0/0	
sgr-0b6d8fae740cf8e9e	All traffic	All	All	0.0.0.0/0	
sgr-09766c467e696f34d	HTTPS	TCP	443	0.0.0.0/0	

At the bottom, there is a warning message: 'Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.'

Backend Security Group:



CONNECT TO EC2(WEB SERVER) EC2 INSTANCE CONNECT:

- Sudo apt update
- Install nginx : sudo apt install nginx
- cd /var/www/html
- sudo nano index.nginx-debian.html
- sudo systemctl restart nginx

CONNECT TO EC2(APP SERVER) EC2 INSTANCE CONNECT:

- sudo nano AWS.pem
- sudo chmod 400 "AWS.pem"
- sudo ssh -i "AWS.pem" ubuntu@backendpublicip
- sudo apt update
- install python : sudo apt install python3-pip -y
- install Flask : pip3 install flask flask-cors --break-system-packages
- create app.py
- Run backend : python3 app.py

Test backend curl [http://publicip:5000/api/message\(status\)](http://publicip:5000/api/message(status))

HTML CODE(index.html):

<!DOCTYPE html>

<html lang="en">

<head>

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>My Website</title>
```

```
<style>
```

```
  * {  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
  }
```

```
  body {  
    font-family: Arial, sans-serif;  
    background: #f4f6f8;  
  }
```

```
  nav {  
    background: #1e3a8a;  
    color: white;  
    padding: 20px 8%;  
  
    display: flex;  
    justify-content: space-between;  
  }
```

```
  nav a {  
    color: white;
```

```
text-decoration: none;
margin-left: 20px;
}
```

```
.hero {
padding: 80px 20px;
text-align: center;

background: linear-gradient(135deg, #2563eb, #7c3aed);
color: white;
}
```

```
.hero h1 {
font-size: 40px;
margin-bottom: 15px;
}
```

```
.form-box {
width: 400px;
max-width: 90%;

margin: 50px auto;
padding: 30px;

background: white;
border-radius: 10px;
box-shadow: 0 5px 20px #ccc;
}
```

```
input {  
    width: 100%;  
    padding: 12px;  
    margin: 10px 0;  
  
    border: 1px solid #ccc;  
    border-radius: 5px;  
}
```

```
button {  
    width: 100%;  
    padding: 12px;  
  
    background: #2563eb;  
    color: white;  
  
    border: none;  
    border-radius: 5px;  
    cursor: pointer;  
}
```

```
button:hover {  
    background: #1d4ed8;  
}
```

```
#apiData {  
    margin-top: 25px;
```

```
padding: 15px;
background: #f1f5f9;
border-radius: 5px;
}
```

```
footer {
padding: 20px;
text-align: center;

background: #1e3a8a;
color: white;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<nav>
```

```
<h2>My Website</h2>
```

```
<div>
```

```
<a href="/">Home</a>
```

```
<a href="#api">API</a>
```

```
</div>
```

```
</nav>
```

```
<section class="hero">
```

<h1>Welcome to My Website</h1>

<p>HTML + CSS + Python Flask + API</p>

</section>

<div class="form-box">

<h2>Contact Us</h2>

<form action="/submit" method="POST">

**<input
type="text"
name="name"
placeholder="Enter your name"
required
>**

**<input
type="email"
name="email"
placeholder="Enter your email"
required
>**

```
<button type="submit">
```

```
  Submit
```

```
</button>
```

```
</form>
```

```
</div>
```

```
<!-- API SECTION -->
```

```
<div class="form-box" id="api">
```

```
  <h2>API Data</h2>
```

```
  <button onclick="fetchData()">
```

```
    Get API Data
```

```
  </button>
```

```
  <div id="apiData">
```

```
    Click the button to fetch data.
```

```
  </div>
```

```
</div>
```

```
<footer>
```

<p>© 2026 My Website</p>

</footer>

<script>

function fetchData() {

document.getElementById("apiData").innerHTML =

"Loading...";

fetch("https://13.60.16.95/users/1")

.then(response => response.json())

.then(data => {

document.getElementById("apiData").innerHTML = `

<p>Name: \${data.name}</p>

<p>Email: \${data.email}</p>

<p>City: \${data.address.city}</p>

<p>Company: \${data.company.name}</p>

`;

})

```
.catch(error => {
```

```
    document.getElementById("apiData").innerHTML =
```

```
        "Unable to fetch API data.";
```

```
    console.error(error);
```

```
});
```

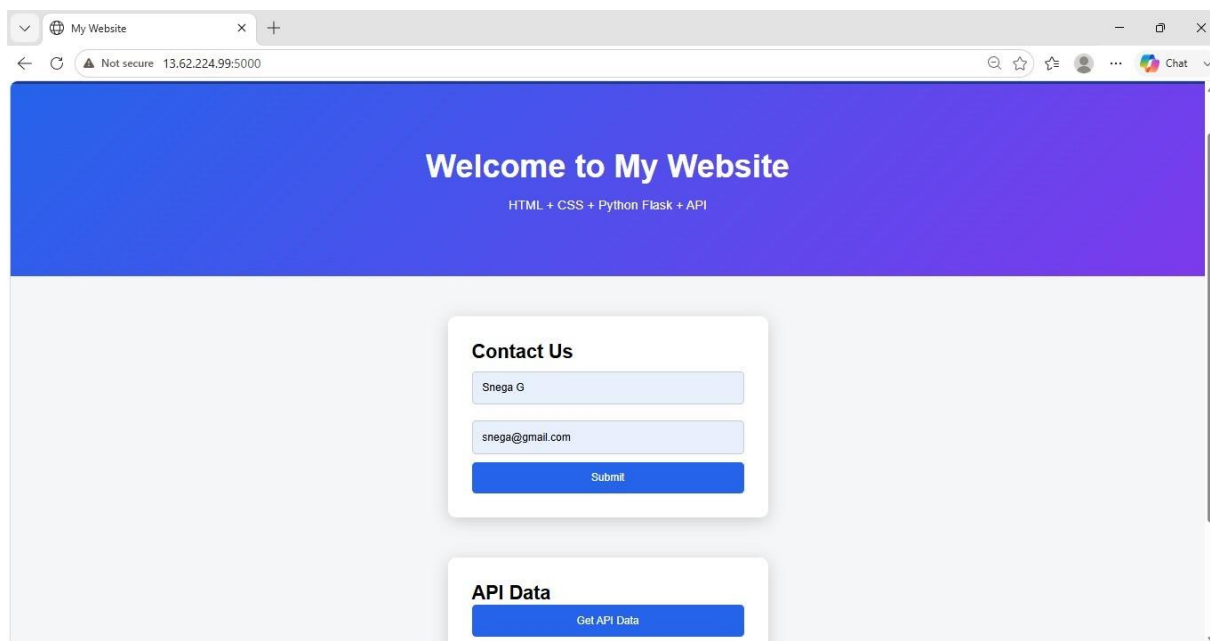
```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Frontend Interface:



Backend Code (app.py):

```
from flask import Flask, render_template, request
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return render_template("index.html")
```

```
@app.route("/submit", methods=["POST"])
```

```
def submit():
```

```
    name = request.form.get("name")
```

```
    email = request.form.get("email")
```

```
    return f"""
```

```
        <h1>Registration Successful</h1>
```

```
        <p>Name: {name}</p>
```

```
        <p>Email: {email}</p>
```

```
        <a href="/">Back to Home</a>
```

```
    """
```

```
if __name__ == "__main__":
```

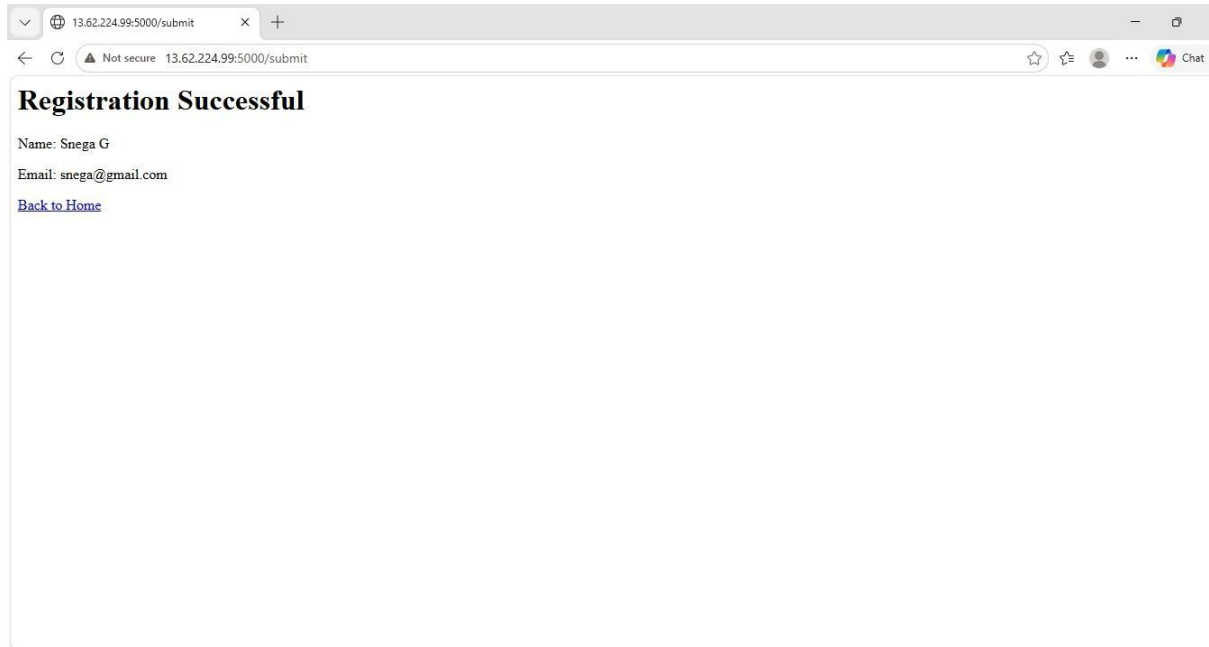
```
    app.run(
```

```
        host="0.0.0.0",
```

port=5000

)

Backend Interface:



Conclusion:

The Registration Web Application successfully demonstrates how a frontend can communicate with a Python Flask Backend. Users can enter their personal information through a simple registration form, and the backend processes and displays the submitted details. The project also demonstrates basic API integration using JavaScript.